Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

StartupMonkey
Autonomous Database Optimiser

| Eric Murray - G00423903 |

| BSc Software Development - ATU Galway |

| https://github.com/EricMurray-e-m-dev/StartupMonkey |

Introduction

The rise of AI-assisted "Vibe Coding" enables developers to ship applications rapidly without deep infrastructure knowledge. Tools like GitHub Copilot and ChatGPT lower the barrier to building production systems, but the operational complexity of running those systems remains. When applications experience traffic growth, database performance degrades silently. Sequential scans replace indexed lookups, connection pools exhaust, and query latency spikes. Solo developers and early-stage startups rarely have the expertise to diagnose these issues before users are affected.

StartupMonkey addresses this gap. It is an autonomous database performance optimisation system built on the MAPE-K (Monitor, Analyse, Plan, Execute with Knowledge) autonomic computing architecture. The system continuously monitors database metrics, detects performance anomalies using a rules-based detection engine, and applies targeted remediations automatically. It supports PostgreSQL, MySQL, and MongoDB, and deploys as a single Docker Compose stack in under five minutes, requiring zero database administration knowledge from the developer.

System Architecture

StartupMonkey is composed of five microservices, all written in Go, with a Next.js dashboard. Services communicate via gRPC for synchronous calls and NATS pub/sub for asynchronous event-driven messaging. The entire system deploys via Docker Compose.

Collector: Polls target databases for performance metrics and streams them to the Analyser via gRPC.

Analyser: Evaluates incoming metrics against configurable thresholds using a rules-based detection engine. Publishes detections to NATS.

Executor: Subscribes to detections and applies targeted remediations autonomously. Supports execute and rollback via the Command Pattern.

Knowledge: Centralised configuration and connection registry, persisted in Redis. All services fetch connection details from Knowledge at runtime.

Dashboard: Next.js interface for monitoring, manual rollback, database onboarding, and real-time action visibility.

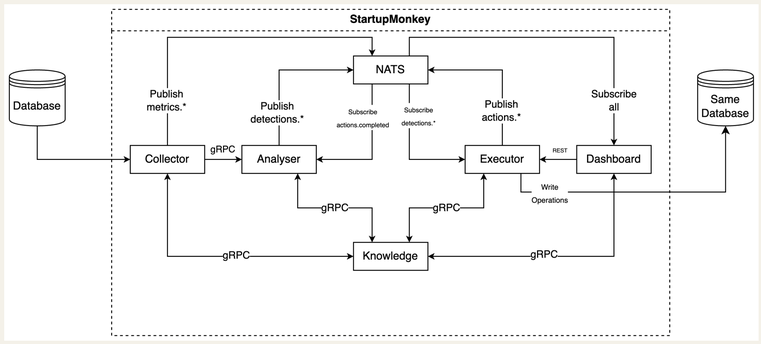

Figure 1

Detection Engine

The Analyser implements a rules-based detection engine that evaluates incoming metrics against configurable thresholds. When a threshold is breached, the Analyser checks the Knowledge service to prevent duplicate detections before publishing to NATS. Seven detectors are implemented: missing index, cache miss, connection pool exhaustion, high latency, long running query, idle transaction, and table bloat. All detectors operate via a database-agnostic adapter interface.

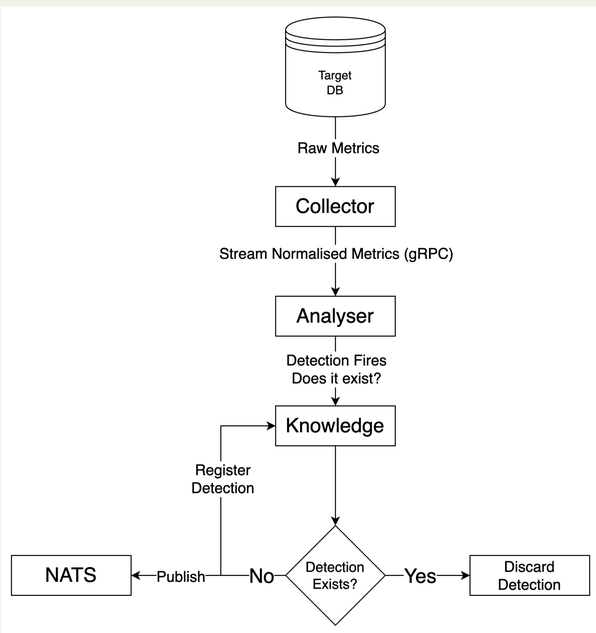

Figure 2

Actions

When a detection is published, the Executor service creates and runs the appropriate action using the Command Pattern, which requires every action to implement execute and rollback methods. Index creation uses CREATE INDEX CONCURRENTLY on PostgreSQL to avoid locking the table during execution. Actions are not all fully autonomous by nature. Some issues, such as schema changes or infrastructure reconfiguration, require developer input. To control this, StartupMonkey exposes three execution modes: Autonomous (actions execute immediately), Approval (actions queue for manual confirmation), and Observe (detections only, no changes applied).

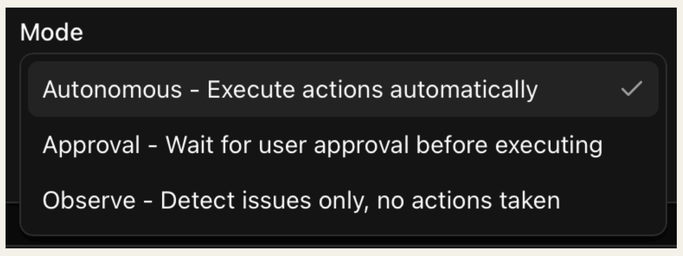

Figure 3

Some remediations are not possible to perform completely autonomously. For example, if we want to deploy a connection pooler like PgBouncer, to reuse connections to help performance. While we can deploy the pooler for the user and config it and keep that side of it autonomous. Some developer intervention will be required for some fixes. However StartupMonkey aims and delivers at keeping intervention to a minimum where possible and doing as much heavy lifting as possible for the developer.

Below is an example of an fully autonomous action, index creation.

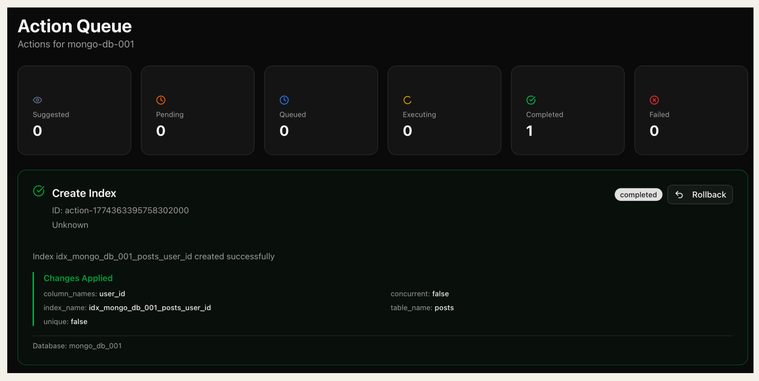

Figure 4

Dashboard

The Next.js dashboard provides real-time visibility into database health, active detections, and action history. On first launch, an onboarding wizard guides the user through registering a database connection and configuring detection thresholds. No database administration knowledge is required. Once connected, the dashboard displays live metrics including connection health, query health, cache hit rate, and storage usage, aggregated into an overall health score. Detections and actions update in real time via NATS subscriptions.

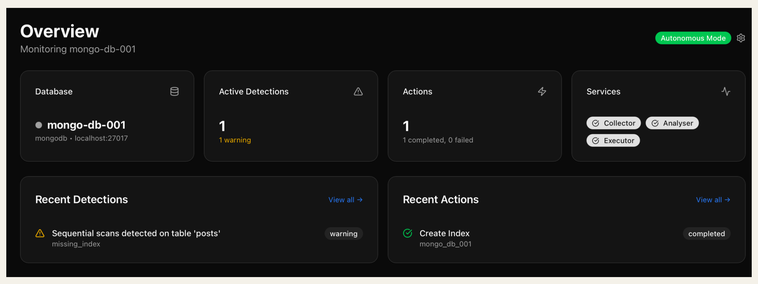

Figure 5

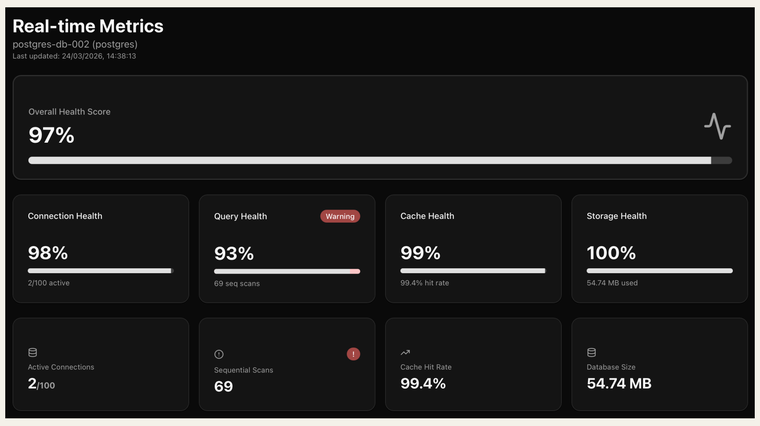

Figure 6

The figures show an overview of all information coming through from all Database sources & focused metrics. Just one DB is configured for simplicity, but complete multi-DB observability and remediation is supported.

Evaluation

System performance was evaluated using a custom lightweight C++ load tester built specifically for this project, targeting a PostgreSQL database with no indexes on a frequently queried column. StartupMonkey autonomously detected the missing index and applied a CREATE INDEX action without manual intervention. The results show a significant improvement across all measured metrics.

Throughput increased from 3,048 to 5,179 requests per second (+70%). Average latency dropped from 15.9ms to 9.2ms (-42%), and P99 latency reduced from 30ms to 17ms (-43%). Zero failed requests were recorded in both runs.

Other detection scenarios including connection pool exhaustion and cache miss also demonstrated measurable remediation improvements. Missing index remediation produced the most consistent and reproducible results and was used as the primary evaluation case.

Heading	Before SM		Heading	After SM		Difference (+/-)
Missing Index Test						
Duration	60.088		Duration	60.024		00.063
Total Reqs	183172		Total Reqs	310850		127678
Successful Reqs	183172		Successful Reqs	310850		127678
Failed Reqs	0		Failed Reqs	0		0
Req/s	3048.411		Req/s	5178.736		2130.325
Latencies (ms)			Latencies (ms)			
Average	15.877		Average	9.164		6.713
Min	4.000		Min	1.000		3.000
Max	126.000		Max	114.000		12.000
P50	15.000		P50	8.000		7.000
P90	20.000		P90	14.000		6.000
P99	30.000		P99	17.000		13.000

Figure 7

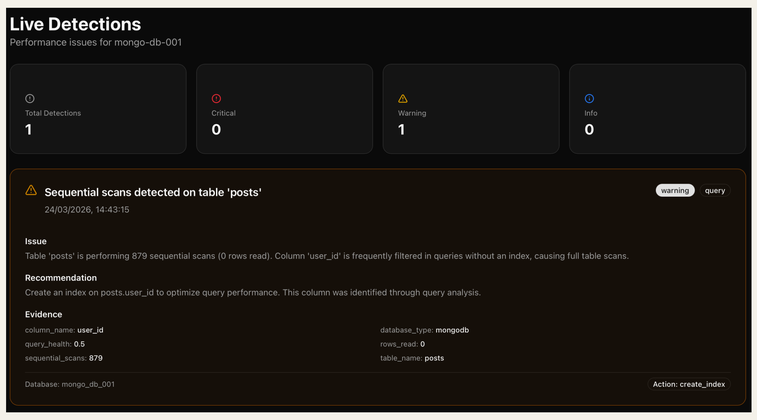

Figure 8

This shows the detection triggering action for a sequential scan threshold break

Conclusion

StartupMonkey demonstrates that autonomous database performance optimisation is achievable at a small scale without machine learning or specialist infrastructure knowledge. The system successfully implements the MAPE-K autonomic computing loop across five independent microservices, with full PostgreSQL support and extensible multi-database architecture covering MySQL and MongoDB.

SOLID principles were applied throughout the codebase, enabling the use of the Adapter, Command, Factory, and Strategy design patterns across every service. This approach allowed additional database support to be introduced without modifying existing logic, validating the extensibility of the design in practice.

A fundamental limitation of any monitoring and remediation tool is that not all database problems are automatically fixable once a system is in production. StartupMonkey can detect and remediate a defined set of known issues, but deeper problems such as schema design flaws or application-level query patterns remain outside its scope.

Future work includes deploying the system against live production databases to gather real-world performance data, which could serve as a training set for ML-based anomaly detection. Further development directions include deeper remediation capabilities, broader database support, and releasing the project as open source to encourage community-driven extension.

Acknowledgements

Supervisor: Dr Douglas Mota Dias, for guidance, support and expertise throughout the project.

The IBM MAPE-K autonomic computing framework (Kephart & Chess, 2003) provided the architectural foundation for the system design.

Claude (Anthropic) was used as the AI coding assistant throughout development.